

Lab7: Insight into Cryptography

Objective

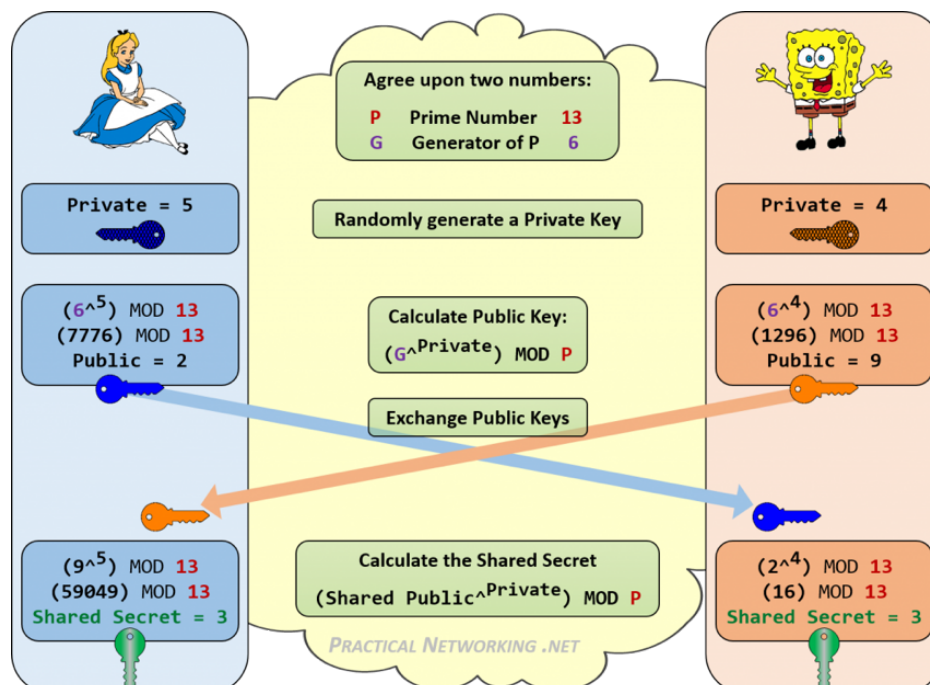
- The aim of this lab is to be familiar with encryption, decryption, and digital signature

Tips

- These labs are expected to take more than the allocated 2 hours. You should try the tasks before the lab. Practice makes perfect.
- Before the lab, you could post your questions in LMO forum, and you are welcome to offer solutions to the questions in LMO.
- During the lab, you could discuss with TAs for your questions. Face to face discussion will have a relatively high efficiency.
- Please use the **university approved calculator** to do all calculations.
- You are recommended to **find a lab-mate** to finish this lab.

A. D-H Shared Secret Generation

- Since the symmetric key cannot be safely exchanged by HTTP, try to negotiate a symmetric key by DH algorithm for password encryption:



2. You are strongly recommended to work with a lab-mate for this process. First, agree on a prime number **P**, then find a primitive root of **P** to use as the generator **G**
(Online tool: <https://owlsmath.neocities.org/Primitive%20Root%20Calculator/calculator>).
3. Then, each of you randomly choose a **private key** and calculate the **public key** for exchange.
4. Next, you should use the received **public key** to generate the shared secret.
5. Finally, verify that both sides have derived the **same shared secret**.

B. RSA Asymmetric Key Generation

1. RSA algorithm uses a key pair mechanism for encryption and digital signature applications.
2. Using the following function, you can easily generate an RSA key pair:

```

1  <?php
2  function myrsa($m, $n){
3      $p = $m * $n;
4      $fip = ($m - 1) * ($n - 1);
5      $a = 7;
6      $d = 0;
7      for($i=1;$i<100;$i++){
8          $d = $i * $fip + 1;
9          if (($d % $a) == 0) {
10             $d = $d / $a;
11             break;
12         }
13     }
14     echo "public key: ".$a.", ".$p."\n";
15     echo "private key: ".$d.", ".$p;
16 }

```

3. Please note that both **m** and **n** should be primitive numbers.

C. RSA Encryption

1. Please ask your lab-mate to share his/her **public key (a, p)**.
2. You can choose any number as the plain information: **x**, which should be smaller than "**p**".
Then, do the encryption operation:

$$x^a \bmod p = X$$

3. Send the **X** to your lab-mate. Let he/she use the **private key** to decrypt the **X**:

$$X^d \bmod p = \text{result}$$

4. Finally, you should check the magic: the decrypted **result** is the same as **x**.

D. Digital Sign

1. You can send a message, **y**, to your lab-mate.
2. How to check the received message is not tamped? You can add a **digital signature** for this message with your **private key**:

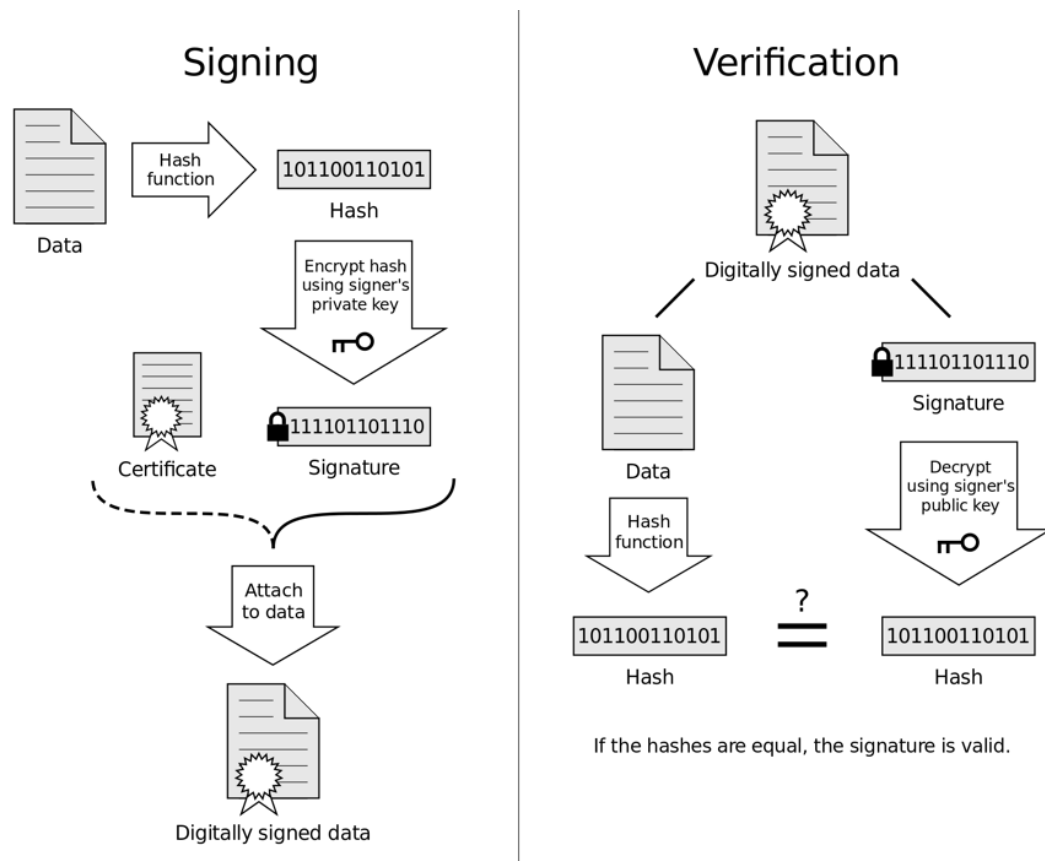
$$y^d \bmod p = \text{signature}$$

3. Your lab-mate then can use your **public key** to decrypt the **signature**:

$$\text{signature}^a \bmod p = \text{result}$$

4. If the **result** is the same as the received message, the message is not tamped.
5. While it may seem unnecessary to decrypt and then compare, in real-world applications the message could be a large file, such as a movie. Signing the entire file is inefficient. Instead, it is common practice to compute a HASH of the file and sign only the HASH. The receiver will calculate the HASH of received file and compare with the decrypted HASH from digital sign.

The whole process is as follows:



6. Hints: $\wedge$ % calculation with big number is not an easy job. You may need to calculate several rounds for big numbers.